

Cheat sheet essenziale - Node, npm e Vite

Obiettivo: riuscire ad aprire il terminale, muoversi nella cartella giusta, capire a cosa servono Node.js e npm, installare dipendenze e avviare un progetto React creato con Vite.

Questa guida usa comandi per **Windows PowerShell**.

1. Terminale: cosa serve sapere

Il terminale serve per dare comandi al computer scrivendoli a mano.

In un progetto React con Vite lo useremo soprattutto per:

- entrare nella cartella del progetto;
- installare le dipendenze;
- avviare il server di sviluppo.

Esempio di terminale PowerShell:

```
PS C:\Users\Marco\code\boolean>
```

La parte `C:\Users\Marco\code\boolean` indica la cartella in cui ti trovi adesso.

2. Comandi base per muoversi

Sapere dove sei:

```
pwd
```

Vedere file e cartelle:

```
ls
```

Entrare in una cartella:

```
cd nome-cartella
```

Tornare indietro di una cartella:

```
cd ..
```

Creare una cartella:

```
mkdir nome-cartella
```

Creare un file:

```
ni nome-file.js
```

Regola pratica: prima di lanciare comandi importanti, controlla sempre di essere nella cartella giusta con `pwd` e `ls`.

3. Percorsi: assoluti e relativi

Un percorso dice dove si trova una cartella o un file.

Percorso assoluto: parte dal disco.

```
C:\Users\Marco\code\boolean
```

Percorso relativo: parte dalla cartella in cui sei adesso.

```
cd mio-progetto  
cd ..  
cd src
```

Per lavorare su un progetto, devi entrare nella sua cartella:

```
cd nome-progetto
```

Poi controlla che ci sia `package.json`:

```
ls
```

4. Node.js

Node.js permette di eseguire JavaScript fuori dal browser.

Nel corso, per ora, ci serve soprattutto per far funzionare strumenti moderni come npm e Vite.

Controllare se Node e' installato:

```
node -v
```

Se compare una versione, per esempio:

```
v22.0.0
```

allora Node e' installato.

Se il comando non viene riconosciuto, probabilmente Node non e' installato o il terminale va riaperto dopo l'installazione.

5. npm

npm e' il gestore di pacchetti incluso con Node.

Serve per:

- installare librerie;
- installare le dipendenze di un progetto;
- lanciare script come `npm run dev`.

Controllare se npm funziona:

```
npm -v
```

Installare tutte le dipendenze di un progetto:

```
npm install
```

Questo comando va lanciato dentro la cartella del progetto, dove si trova `package.json`.

6. package.json

`package.json` e' il file che descrive un progetto Node/npm.

In un progetto React con Vite contiene soprattutto:

- nome del progetto;
- script da lanciare;
- dipendenze installate.

Esempio semplificato:

```
{
  "name": "my-react-app",
  "private": true,
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "latest",
    "react-dom": "latest"
  },
  "devDependencies": {
    "vite": "latest",
    "@vitejs/plugin-react": "latest"
  }
}
```

La parte piu' importante da riconoscere per ora e' **scripts**:

```
"scripts": {
  "dev": "vite"
}
```

Questo significa che puoi avviare Vite con:

```
npm run dev
```

7. Dipendenze

Le dipendenze sono librerie usate dal progetto.

Esempi:

- **react**
- **react-dom**
- **vite**

Quando scarichi o cloni un progetto, la cartella **node_modules** spesso non c'e'. E' normale.

Per ricrearla:

```
npm install
```

Non modificare a mano **node_modules**.

8. Vite

Vite e' lo strumento che usiamo per creare e avviare progetti frontend moderni.

Per noi, in questa fase, Vite serve a:

- creare un progetto React;
- avviare un server locale;
- aggiornare la pagina quando salvi i file.

Creare un nuovo progetto React con Vite:

```
npm create vite@latest
```

Durante la creazione scegli:

```
Framework: React  
Variant: JavaScript
```

Poi entra nella cartella del progetto:

```
cd nome-progetto
```

Installa le dipendenze:

```
npm install
```

Avvia il progetto:

```
npm run dev
```

Il terminale mostrera' un indirizzo simile:

```
http://localhost:5173/
```

Aprilo nel browser.

9. Sequenza da ricordare

Per creare un nuovo progetto React:

```
npm create vite@latest  
cd nome-progetto  
npm install  
npm run dev
```

Per riprendere un progetto già creato:

```
cd nome-progetto  
npm install  
npm run dev
```

Nota: `npm install` serve quando apri il progetto per la prima volta, quando manca `node_modules`, o quando sono cambiate le dipendenze.

10. Se qualcosa non funziona

Controlla di essere nella cartella giusta:

```
pwd  
ls
```

Controlla che ci sia `package.json`:

```
ls
```

Installa le dipendenze:

```
npm install
```

Avvia il progetto:

```
npm run dev
```

Errori comuni:

- `node` non viene riconosciuto: Node non è installato o devi riaprire il terminale.
- `npm` non viene riconosciuto: npm non è disponibile, controlla l'installazione di Node.
- `Missing script: dev`: probabilmente non sei nella cartella giusta o il progetto non è un progetto Vite.
- `Cannot find module`: probabilmente devi lanciare `npm install`.

11. Mini glossario

- **Terminale:** finestra dove scrivi comandi.
- **PowerShell:** terminale/shell usata su Windows.
- **Cartella corrente:** cartella in cui ti trovi nel terminale.
- **Node.js:** programma che esegue JavaScript fuori dal browser.
- **npm:** strumento per installare pacchetti e lanciare script.
- **package.json:** file con informazioni, script e dipendenze del progetto.
- **Dipendenza:** libreria usata dal progetto.
- **Vite:** strumento per creare e avviare progetti frontend.
- **React:** libreria per costruire interfacce usando componenti.